

# Vibration Analysis System using MSA301 Triaxial Accelerometer & STM32F401RE

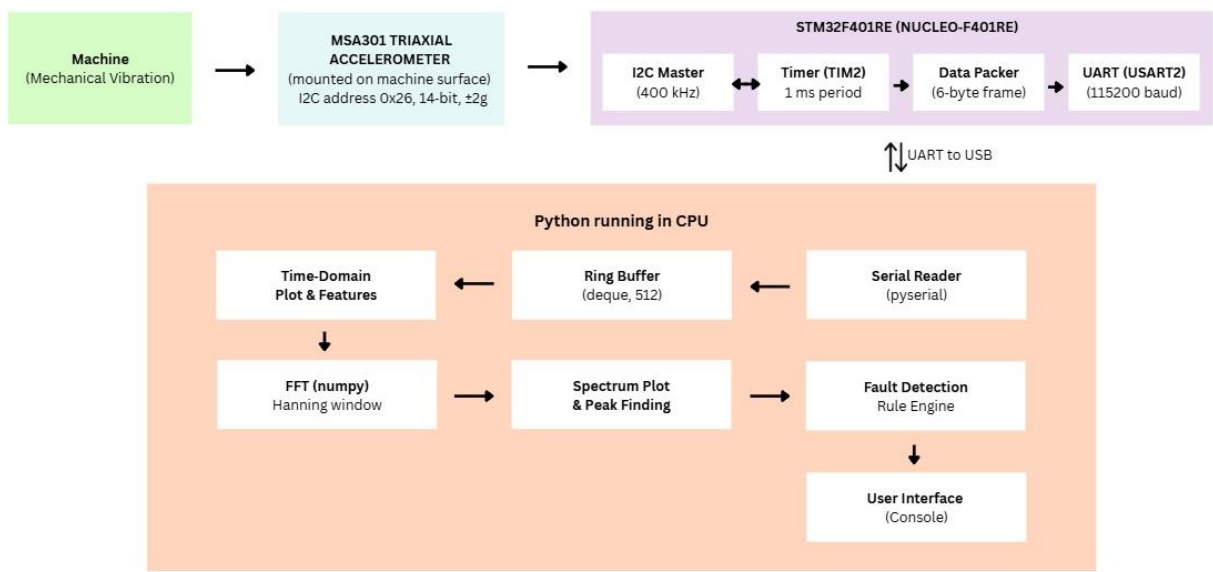
## Project Objective

To build a real-time vibration monitoring system for machinery health. The system captures 3-axis acceleration data using an MSA301 sensor interfaced with an STM32 microcontroller, then extracts time-domain metrics (RMS, peak, crest factor, kurtosis) and performs frequency-domain analysis (FFT) to detect anomalies, diagnose mechanical faults, and predict failures.

## System Components

| Component       | Model / Part                    | Key Specifications                                                     |
|-----------------|---------------------------------|------------------------------------------------------------------------|
| Microcontroller | STM32 Nucleo-64 (NUCLEO-F401RE) | ARM Cortex-M4, I2C, 3.3V logic, 5V tolerant                            |
| Accelerometer   | Adafruit MSA301                 | I2C address 0x26, 14-bit resolution, $\pm 2/4/8/16g$ , up to 1 kHz ODR |
| Development     | STM32CubeIDE + HAL libraries    | I2C, USART                                                             |

## System Architecture



- Machine → Accelerometer – mechanical vibration coupling
- Accelerometer → I2C Master – I2C bus (SCL/SDA) connection
- Timer (TIM2) → I2C Master – timer interrupt triggers data reading
- I2C Master → Data Packer – raw 14-bit values to 16-bit packing
- Data Packer → UART (USART2) – 6-byte frames to transmit
- UART → UART to USB – physical USB-UART bridge
- UART to USB → Serial Reader (Python) – virtual COM port data
- Serial Reader → Ring Buffer – raw binary data into a sliding window
- Ring Buffer → Time-Domain Plot & Features – compute RMS, peak, crest, kurtosis
- Ring Buffer → FFT (NumPy) – take a window (512 samples) for frequency analysis
- FFT → Spectrum Plot & Peak Finding – magnitude spectrum
- Time-Domain Features + Spectrum → Fault Detection (Rule Engine) – combine evidence
- Fault Detection → User Interface – display alerts / colour-coded status

## Hardware Wiring (I2C)

| MSA301 Pin | NUCLEO-F401RE Pin (ARDUINO®) | STM32 Pin | Function                              |
|------------|------------------------------|-----------|---------------------------------------|
| VIN        | CN6 pin 5 (+5V)              | —         | 5V power (on-board regulator to 3.3V) |
| GND        | CN6 pin 6                    | —         | Ground                                |
| SCL        | CN10 pin 3                   | PB8       | I2C1_SCL                              |
| SDA        | CN10 pin 5                   | PB9       | I2C1_SDA                              |
| TX         | CN10 pin 22                  | PB2       | USART2_TX                             |
| RX         | CN10 pin 31                  | PB3       | USART2_RX                             |

## Software Implementation

### 1. STM32 Firmware (Cube IDE)

The firmware configures the MSA301 for **1000 Hz output data rate (ODR)**, uses a **timer interrupt** to read the sensor data and sends raw 16-bit signed integers over **UART** in binary format.

Key Configuration

1. I2C1 – 400 kHz (Fast Mode)
2. USART2 – 115200 baud, 8N1 (binary transmission)
3. TIM2 – 1 ms period (1000 Hz interrupt)

### Register Settings for MSA301

| Register   | Value | Description                |
|------------|-------|----------------------------|
| CTRL_REG1  | 0x07  | Active mode, ODR = 1000 Hz |
| RANGE_SEL  | 0x00  | ±2g full scale             |
| RESOLUTION | 0x04  | 14-bit resolution          |

### 2. Python Script

The Python script reads the binary UART data, buffers 512 samples, updates a live time-domain plot, computes the FFT spectrum, and calculates real-time features (RMS, peak, crest factor, kurtosis). It also implements simple fault detection thresholds.

```
# ----- CONFIGURATION -----  
PORT = 'COM5'           # Change to your port  
BAUD = 115200  
SAMPLE_RATE = 1000      # Hz  
BUFFER_SIZE = 512       # samples for one FFT  
SCALE = 0.000244        # g per LSB (14-bit, ±2g)
```

## Data Flow & Processing

1. STM32 reads MSA301 every 1 ms (1000 Hz) in a timer ISR.
2. Raw 14-bit values are converted to 16-bit signed integers and sent over UART (6 bytes per sample, little-endian).
3. Python script:
  - a. Reads binary data from serial port.

- b. Maintains a sliding window of 512 samples.
- c. Plots live acceleration waveform.
- d. Computes RMS, peak, crest factor, kurtosis.
- e. Performs FFT and displays spectrum.
- f. Flags anomalies based on predefined thresholds.

Fault Detection Methodology

1.Time-Domain Features:

| Feature      | Formula                                      | Normal Range  | Fault Indication                                         |
|--------------|----------------------------------------------|---------------|----------------------------------------------------------|
| RMS (g)      | $\sqrt{(1/N \sum x_i^2) \lim N-1}$           | < 0.1 g       | > 0.5 g → excessive vibration (unbalance, looseness)     |
| Peak (g)     | $\max( x_i )$                                | –             | High impacts (bearing damage, gear tooth breakage)       |
| Crest Factor | Peak/RMS                                     | 3 – 5         | > 5 → impulse-type faults (rolling element bearing wear) |
| Kurtosis     | $((1/N) \sum (x_i-\mu)^4)/\sigma^4 \lim N-1$ | ~3 (Gaussian) | > 5 → early bearing fault; > 10 → severe damage          |

2. Frequency-Domain Features (FFT)

| Fault Type   | Frequency Signature                      | Example                                        |
|--------------|------------------------------------------|------------------------------------------------|
| Unbalance    | 1× rotational frequency (fundamental)    | Peak at shaft speed (e.g., 30 Hz for 1800 RPM) |
| Misalignment | 2× rotational frequency (2nd harmonic)   | High peak at twice shaft speed                 |
| Looseness    | Multiple harmonics (1×, 2×, 3×, 4×, ...) | Several peaks at integer multiples             |

Conclusion

The system captures three-axis acceleration data at 1 kHz, streams it via UART to a PC, and performs comprehensive signal processing using Python.

1. Reliable data acquisition – The STM32 firmware correctly initialises the MSA301, reads data at precise 1 ms intervals using a timer interrupt, and transmits raw binary frames without loss.
2. Time-domain feature extraction – RMS, peak acceleration, crest factor, and kurtosis are computed in real time, providing immediate insight into vibration severity and impact activity.
3. Frequency-domain analysis – The Python script performs a 512-point FFT with a Hamming window, producing a clear magnitude spectrum that reveals dominant frequencies, harmonics, and potential fault signatures.
4. User-friendly visualisation – Real-time plots of the acceleration waveform and FFT spectrum, together with fault indicators, allow the operator to monitor machine health instantly.

In summary, the developed system provides a cost-effective, scalable, and informative platform for predictive maintenance. It successfully bridges embedded data acquisition with PC-based analytics, offering a solid foundation for further industrial implementation.